2026 算法导论第一次作业

1 第一题

考虑下面的基本问题。给定由 n 个整数 $A[1], A[2], \ldots, A[n]$ 组成的数组 A。你想输出一个二维的 $n \times n$ 数组 B，其中 $B[i, j]$（$i < j$）包含数组项 $A[i]$ 到 $A[j]$ 的和，即

$$A[i] + A[i+1] + \cdots + A[j].$$

只要 $i \geq j$，数组项 $B[i, j]$ 的值不用指定，因此不必输出这些值。

这里是求解这个问题的一个简单算法：

```
For i = 1, 2, ..., n − 1
    For j = i + 1, i + 2, ..., n
        把数组元素从 A[i] 加到 A[j]
        把这个结果存储到 B[i, j]
    Endfor
Endfor
```

(a) 针对某些你应该选择的函数 f，对这个算法给出一个形为 $O(f(n))$ 的在规模为 n 的输入上运行时间的界（即这个算法所执行的操作个数的界）。

(b) 对同一个函数 f，证明算法在规模为 n 的输入上的运行时间也是 $\Omega(f(n))$（这证明了在运行时间上的一个 $\Theta(f(n))$ 的渐近紧界）。

(c) 尽管你在 (a) 和 (b) 中分析的算法是求解这个问题的最自然的方法——毕竟，它仅仅在数组 B 的有关项上迭代，并对每项填入一个值——但是它包含了某些非常不必要的成分，导致了它的效率低下。给出一个求解这个问题的不同算法，使得它具有渐近更好的运行时间。换句话说，你应该设计一个具有 $O(g(n))$ 运行时间的算法，其中

$$\lim_{n \to \infty} \frac{g(n)}{f(n)} = 0.$$

【解答】

(a)

我们将得到每个 $B[i, j]$ 所需的加法次数绘制成矩阵，可得

$$\begin{bmatrix} 0 & 1 & 2 & 3 & 4 & \cdots & n-2 & n-1 \\ 0 & 0 & 1 & 2 & 3 & \cdots & n-3 & n-2 \\ 0 & 0 & 0 & 1 & 2 & \cdots & n-4 & n-3 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & \cdots & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & \cdots & 0 & 0 \end{bmatrix}$$

因此有

$$\begin{aligned} \frac{n(n-1)}{2} + \frac{(n-1)(n-2)}{2} + \cdots + 1 &= \sum_{k=1}^{n-1} \frac{k(k+1)}{2} \\ &= \frac{1}{2}\left(\sum_{k=1}^{n-1} k^2 + \sum_{k=1}^{n-1} k\right) \\ &= \frac{1}{2}\left(\frac{(n-1)n(2n-1)}{6} + \frac{(n-1)n}{2}\right) \\ &= \frac{n(n-1)(n+1)}{6}. \end{aligned}$$

所以该算法的时间复杂度为

$$O(f(n)) = O(n^3).$$

(b)

由 (a) 可知，

$$T(n) = \frac{n(n-1)(n+1)}{6}.$$

因此当 n 足够大时，存在正常数 $c > 0$ 与 n_0，使得对所有 $n \geq n_0$，都有

$$T(n) \geq cn^3.$$

于是可得

$$T(n) = \Omega(n^3).$$

综上，

$$T(n) = \Theta(n^3).$$

(c)

平凡算法存在的问题在于有大量重复计算，导致了额外开销。我们可以像改进斐波那契数列的计算一样，利用前面已经得到的结果，在原有基础上计算新的 $B[i, j]$，而不必每次都从头开始求和。

伪代码如下：

```
For i = 1, 2, ..., n − 1
    sum = A[i]
    For j = i + 1, i + 2, ..., n
        sum = sum + A[j]
        B[i, j] = sum
    Endfor
Endfor
```

这样我们就可以用 sum 记录每一行 $B[i,j]$ 的前一位 $B[i,j-1]$ 的数值，在此基础上加上 $A[j]$ 便可以得到 $B[i,j]$，节省了计算开销。

此时操作次数复杂度为

$$g(n)=\sum_{i=1}^{n-1}(n-i)=\sum_{k=1}^{n-1}k=\frac{n(n-1)}{2}=O(n^2).$$

即 $O(g(n))=O(n^2)$，且满足

$$\lim_{n\to\infty}\frac{g(n)}{f(n)}=\lim_{n\to\infty}\frac{\frac{n(n-1)}{2}}{\frac{n(n-1)(n+1)}{6}}=\lim_{n\to\infty}\frac{3}{n+1}=0.$$

2 第二题

你正在对不同的玻璃瓶样品做强度测试以确定它们从多高的高度掉下来而仍旧不碎. 对某类特定的瓶子如下设计这样一个实验. 你有一个具有 n 个横挡的阶梯, 并且你想找出最高的横挡, 能使一个瓶子的样品从那里下落而不被摔碎. 我们把它称为“最高的安全横挡”.

尝试二分搜索可能是一种自然的想法: 使一个瓶子从中间的横挡下落, 看看它是否摔碎, 然后依赖于这个结果递归地从 $n/4$ 横挡或者 $3n/4$ 的横挡进行尝试. 但是这个办法有缺点, 在找到答案之前, 你可能摔碎了一大堆瓶子.

如果你的主要目标是保护瓶子, 另一方面, 你可以尝试下面的策略. 从第一个横挡开始让瓶子下落, 然后第二个横挡, 继续下去, 每次向上爬一个高度直到瓶子摔碎为止. 以这种方式, 你只需要一个瓶子——在它摔碎的时刻, 你得到正确的答案——但是你可能不得不把它下落 n 次 (而不是在二分搜索求解时的 $\log n$ 次).

于是, 这里是某种权衡: 似乎是如果你愿意打碎更多的瓶子, 你就可以执行更少的下落. 为了更好地理解这种权衡在量级上是怎样起作用的, 让我们考虑给定固定的 $k \geq 1$ 个瓶子的“预算”, 怎样来运行这个实验. 换句话说, 你必须确定正确的答案——最高的安全横挡——并且在这个过程中至多可以用 k 个瓶子.

(a) 假设只给你 $k = 2$ 个瓶子. 描述一种找到最高安全横挡的策略, 对某个比线性函数增长更慢的函数 $f(n)$, 要求一个瓶子至多落 $f(n)$ 次 (换句话说, 应该满足 $\lim_{n\to\infty} f(n)/n = 0$).

(b) 假设你的瓶子数的预算是某个给定的 $k > 2$. 描述一种至多使用 k 个瓶子找到最高安全横挡的策略. 如果 $f_k(n)$ 表示依照你的策略需要下落瓶子的次数, 那么函数 $f_1, f_2, f_3, \ldots$ 应该有这样的性质, 每个函数渐近增长比前面的函数要慢: 对每个 k, $\lim_{n\to\infty} f_k(n)/f_{k-1}(n) = 0$.

【解答】

(a)

我根据这个题目在 LeetCode 上找到了母题 Super Egg Drop，阅读了官方题解后我觉得可以迁移到这个问题上：

如题目所述，因为只有两个瓶子，我们无法使用二分查找，这会消耗大量瓶子；而自下而上依次尝试，效率低，无法满足 $\lim_{n\to\infty} f_k(n)/f_{k-1}(n) = 0$，因此我们采取如下策略将分块和逐个尝试两种方法结合：

当 $k = 2$ 时，取步长 $t = \lceil \sqrt{n} \rceil$。

先用第一个瓶子在横挡 $t, 2t, 3t, \ldots$ 处测试，直到第一次摔碎；然后用第二个瓶子在前一个区间内顺序查找。

最坏情况下落次数为：

$$\left\lceil \frac{n}{t} \right\rceil + t - 1 \approx \sqrt{n} + \sqrt{n} - 1 = 2\sqrt{n} - 1,$$

所以：

$$f(n) = O(\sqrt{n}).$$

而且满足：

$$\lim_{n\to\infty} \frac{f(n)}{n} = \lim_{n\to\infty} \frac{O(\sqrt{n})}{n} = 0.$$

(b)

对一般的 $k > 2$，采用递归分块策略。把 n 个横挡分成块，每块大小取：

$$m = \left\lceil n^{(k-1)/k} \right\rceil.$$

先用 1 个瓶子测试各块末端，确定答案所在的块；再在该块内用剩下的 $k-1$ 个瓶子递归查找。最坏情况下的抛掷次数满足如下递推关系：

$$f_k(n) \le \left\lceil \frac{n}{m} \right\rceil + f_{k-1}(m).$$

为了直观说明这个递推过程是如何达到平衡的，我们从底层逐个分析：

- **当 $k = 1$ 时（1 个瓶子）：** 只能从下往上逐级测试，最坏代价为 $f_1(n) = n = O(n^1)$。
- **当 $k = 2$ 时（2 个瓶子）：** 划分块大小 $m = \lceil n^{1/2} \rceil$。第一阶段抛掷 $\lceil n/m \rceil \approx n^{1/2}$ 次；第二阶段转化为 $k = 1$ 的问题，在规模为 m 的块中查找，代价为 $f_1(m) \approx m = n^{1/2}$ 次。两阶段代价平衡，总代价为 $f_2(n) = O(n^{1/2}) + O(n^{1/2}) = O(n^{1/2})$。
- **当 $k = 3$ 时（3 个瓶子）：** 划分块大小 $m = \lceil n^{2/3} \rceil$。第一阶段抛掷 $\lceil n/m \rceil \approx n/n^{2/3} = n^{1/3}$ 次；第二阶段转化为 $k = 2$ 的问题，在规模为 m 的块中查找，代入上面 $k = 2$ 的结论，代价为 $f_2(m) = O(m^{1/2}) = O((n^{2/3})^{1/2}) = O(n^{1/3})$ 次。两阶段代价再次完美平衡，总代价为 $f_3(n) = O(n^{1/3}) + O(n^{1/3}) = O(n^{1/3})$。
- **当 $k = 4$ 时（4 个瓶子）：** 划分块大小 $m = \lceil n^{3/4} \rceil$。第一阶段抛掷 $\lceil n/m \rceil \approx n/n^{3/4} = n^{1/4}$ 次；第二阶段转化为 $k = 3$ 的问题，在规模为 m 的块中查找，代入上面 $k = 3$ 的结论，代价为 $f_3(m) = O(m^{1/3}) = O((n^{3/4})^{1/3}) = O(n^{1/4})$ 次。总代价为 $f_4(n) = O(n^{1/4}) + O(n^{1/4}) = O(n^{1/4})$。

由此规律不断向上递推可知，公式 $m = \lceil n^{(k-1)/k} \rceil$ 的精妙之处在于它能始终保持前后两阶段的测试代价处于同一量级。对于任意给定的 k，其最坏情况下的总下落次数为：

$$f_k(n) = O(n^{1/k}).$$

因此，前后两个函数渐近增长率的比值极限为：

$$\lim_{n\to\infty}\frac{f_k(n)}{f_{k-1}(n)}=\lim_{n\to\infty}\frac{O(n^{1/k})}{O(n^{1/(k-1)})}=\lim_{n\to\infty}O\left(n^{\frac{1}{k}-\frac{1}{k-1}}\right).$$

因为 $k>k-1>0$，所以 $\frac{1}{k}-\frac{1}{k-1}<0$。即随着 $n\to\infty$，该极限趋于 0：

$$\lim_{n\to\infty}\frac{f_k(n)}{f_{k-1}(n)}=0.$$

这证明了依照此策略，每增加一个瓶子的预算，函数的渐近增长都严格比前面的函数要慢。